

Seq 2 Seq MODELS

Example of Conditional Language Model

Sequence to sequence (Seq 2 Seq) models are neural architectures that map an input sequence to an output sequence of variable length.

- Machine Translation
- Text Summarization.

→ Encoder - Decoder

- The core of Seq 2 Seq
- Both Encoder and Decoder are usually RNN based (LSTM/GRU) in traditional setups.

• Encoder

- Reads the input sequence $(x_1, x_2, \dots, x_n)$
- Produces a context vector C summarizing all information

$$h_t = f(x_t, h_{t-1})$$

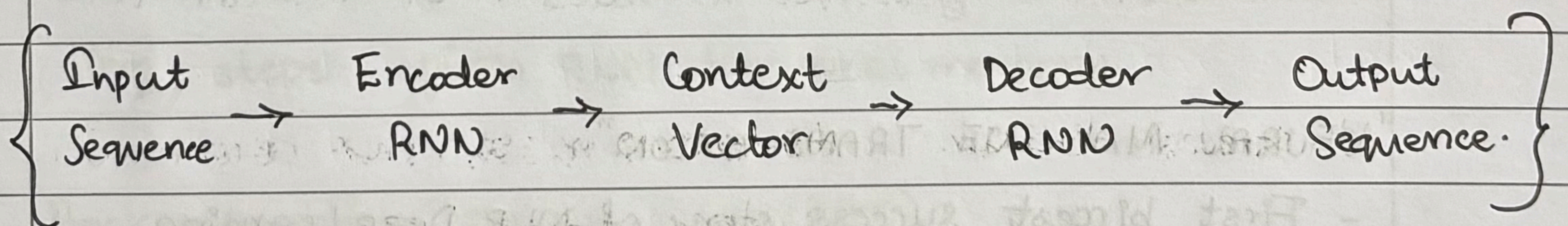
$$C = h_T$$

• Decoder

- Takes the context vector C and generates the o/p sequence
- Each step depends on previous predictions & the context

$$s_t = f(y_{t-1}, s_{t-1}, C)$$

$$P(y_t | y_{1:t}, C) = \text{softmax}(W s_t)$$



The model learns to maximise the likelihood of the target sequence given the input

$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t | y_{1:t}, x)$$

Teacher Forcing:

Instead of feeding predicted y_{t-1} , we feed the true prev tokens during training

Why

Teacher forcing? It accelerates convergence and avoids compounding errors early in training.

Drawback: During inference, model must rely on its own predictions - this creates exposure bias.

Limitations of Vanilla Seq2Seq

- Fixed context vector for long sequences.
- Error Propagation: mistakes accumulate during training.

→ Decoding Strategies in Seq2Seq Models

Once model is trained, during inference, we must decode the most likely output sequence.

We want the most probable sequence:

$$\hat{y} = \underset{y}{\operatorname{argmax}} P(y|x)$$

But enumerating all possible sequences is infeasible → we use heuristic decoding strategies.

While generating next token, based on the o/p probability distribution, trade-off b/w two factors need to be handled:

1. Quality: Responses generated → coherent and accurate
2. Diversity: Responses generated → creative and diverse.

Strategies:

① Greedy Decoding: At each time step, pick the word with highest probability.

$$y_t = \operatorname{argmax} P(y_t | y_{1:t}, x)$$

x Might lose globally optimal sequences

② Beam Search Decoding: Maintains k -best partial sequences (beam width = k) at each step instead of one.

1. Start with each token $\langle s \rangle$
2. At each step, expand all beams with next token probabilities
3. Keep top k sequences by cumulative probability
4. Continue until end token $\langle \text{eos} \rangle$

✓ Balances exploration and exploitation

✗ Computationally expensive for large k .

③ Top- k sampling: Choose top k -most probable tokens re-normalize probabilities and sample from them.

- Restricts randomness to meaningful words.

✓ Improves diversity while retaining coherence.

✗ Choosing k is tricky.

④ Top- p (Nucleus) Sampling: Select smallest set of tokens whose cumulative probability $\geq p$ (eg: 0.8) and sample from that set.

$$\sum_{w \in V^{(p)}} P(w | c, w_{<t}) \geq p$$

- Truncate the distribution to remove the very unlikely tokens.

⑤ Temperature Sampling: Scales logits before Softmax

We don't truncate the distribution, instead reshape it.

Normal

Softmax: $P(t_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$

Softmax

with Temp: $P(t_i) = \frac{\exp(z_i / \tau)}{\sum_j \exp(z_j / \tau)}$

$T=1$, normal softmax

$T>1$, Flatter

$T<1$, Sharper

→ Limitations of RNN based Seq2Seq Models

1. Bottle Neck Problem (Information Compression)

- The entire input sequence must be represented as a single fixed-length vector (c)
- For long sentences, this leads to information loss.

2. Lack of Parallelization

- RNN processes one token at a time; $h_t = f(h_t, h_{t-1})$
- Cannot capture / compute in parallel across timesteps.

Solution?

ATTENTION MECHANISM

"Don't compress everything to one vector, let the model learn where to look".

Attention mechanism allows the decoder to look at all encoder states when generating each o/p, rather than relying only on the final hidden state.

Instead of using a single context vector C , the decoder computes a dynamic context vector for each output step:

$$C_t = \sum_{i=1}^{T_x} \alpha_{t,i} h_i$$

where:

h_i = encoder hidden state for input token i

$\alpha_{t,i}$ = attention weight (how much attention to pay to each i/p word)

complete
Intuition

let's say we are translating:

"Ich liebe dich" $\rightarrow$ "I love you"

At the time of generating the word "love", the decoder shouldn't rely on just one vector - it should "attend" mostly to the word "liebe".

- Looks at all encoder outputs
- Decides how much attention each should get
- Combines them into a weighted context vector.

1. Attention uses 3 vectors for each token:

- Query (Q): What we're looking for: Current decoder state
- Key (K): What each i/p word represents: Tag of encoder o/p
- Value (V): What each i/p word holds: Actual encoder info

Mechanism to focus on relevant parts of another sequence.

2. Scoring Relevance: How relevant each encoder word (key) is to the current decoder state (query).

Score formula (single Q): $\text{score}(Q, K_i) = Q \cdot K_i^T$

or more commonly in Scaled Dot-Product Attention:

$\text{score}(Q, K_i) = \frac{Q \cdot K_i^T}{\sqrt{d_k}}$

$\sqrt{d_k}$ - dimension of Key vector.

3. Softmax to normalize: Convert raw scores into a probability distribution.

$$\alpha_i = \text{softmax}(\text{score}(Q, K_i))$$

α_i : how much attention the decoder should pay to the i^{th} encoder word

4. Weighted Sum (Context vector): Use attention weights to combine values

$$\text{Context} = \sum_i \alpha_i V_i$$

//_

Context vector is passed through decoder at each step to generate the next word.

5. Mathematical Form (Matrix Form)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- ✓ Overcomes Bottleneck : dynamic attention vectors
- ✓ Improve long sequence handling
- ✓ Interpretability: Attention weights visualize which inputs influenced outputs
- ✓ Better Gradient Flow
- ✓ Helps with Vanishing Gradient Problem

→ Connection Chain to Remember

1. Text can't be represented numerically - One Hot Encoding

→ Introduces sparse vector

2. Word2Vec solves ↑ OHE vectors having no meaning / similarity

→ Sequence / context of words are not learnt.

3. RNN adds recurrence ↑ processes words sequentially with hidden state

→ Forgets long term information

4. LSTM adds memory cells and gates ↑ to control information flow.

→ Cannot handle input-output sequence mapping

5. Seq2Seq fixes ↑ this by adding two LSTMs (encoder

compresses input, decoder generates output)

→ Max fixed size context bottleneck

6. Attention ↑ adds weighted focus on relevant encoder states dynamically

Improves handling of long inputs, interpretable focus across words.

→ Still sequential processing...

WHAT TO DO?